Network Security

(CS-6008)

Secure Chat Assignment – Report

Sameer Arvind Patil (23B1035)
Abhi Jain (23B0903)

Contents

1 Experimental Topology **2**

2 Phase 1 – Baseline Chat **3**

2.1 Implementation 3

2.2 Verification 3

3 Phase 2 – Client–Server Confidentiality **4**

3.1 DH and AES-GCM Design 4

3.2 Required Verification 5

3.3 Wireshark Capture 5

3.4 Integrity Test 6

3.5 MITM Attack 6

4 Phase 3 – Server Authentication via PKI **8**

4.1 PKI Setup 8

4.2 Authenticated Handshake Design 8

4.3 Verification and Proxy Re-attempts 9

4.4 Phase 3 Result 12

5 Phase 4 – End-to-End Encryption Between Clients **12**

5.1 Design 12

5.2 Verification 12

6 Phase 5 – Forward Secrecy **13**

6.1 Rekey Design 13

6.2 Rotation Verification 14

6.3 What Forward Secrecy Adds 14

A Generative AI Prompt Log **15**

1 Experimental Topology

The required four-node setup is utilized for attack demonstrations, consisting of one server, two clients, and a separate Mallory node for the MITM proxy. Because our system architecture is fully open and supports connecting any number of clients and proxies dynamically, we did not rely on a single, static set of virtual machines for all phases.

Instead, testing and packet captures were conducted across a combination of different Docker containers, virtual machines, and physical laptops. Utilizing Docker containers provided distinct, routable IP addresses on a virtual bridge network (e.g., the `172.28.x.x` subnet), which completely fulfills the assignment's requirement to demonstrate authentic network traffic and packet interception without relying solely on local loopback.

We use `Makefile` targets to launch the server, clients, and proxy. The server listens on a specified `ServerIP:ServerPort`, while the clients connect to that address. Mallory is configured with its own distinct IP and the server's IP for MITM operation. The system can be run with any combination of IP addresses and ports by passing the relevant arguments to the `Makefile` targets.

To assist with grading and correlating the Wireshark captures with our network layout, below is the representative topology mapping of the node roles and IP addresses used during the containerized network demonstrations:

Node Role	**IP Address (Docker Bridge Network)**
Server (S)	`172.28.0.12`
Client 1 ($C1$)	`172.28.0.10`
Client 2 ($C2$)	`172.28.0.11`
Mallory / Proxy (M)	`172.28.0.18`

Note: While the cryptographic and wire-level verifications (Wireshark) were strictly executed across the isolated container network shown above, some UI demonstration screenshots may depict `127.0.0.1` as they were captured during parallel rapid-iteration testing on local host interfaces.

```
[Server] Commands:
          /login username    Log in to the server
          /chat username     Switch the local chat target
          /e2e username      Establish an end-to-end session
          @username message  Send a message and switch target
          /who               Show online users
          /clear             Clear the terminal output
          /help              Show this help
          /quit              Disconnect and exit
```

Figure 1: Command-line tools inside the chat

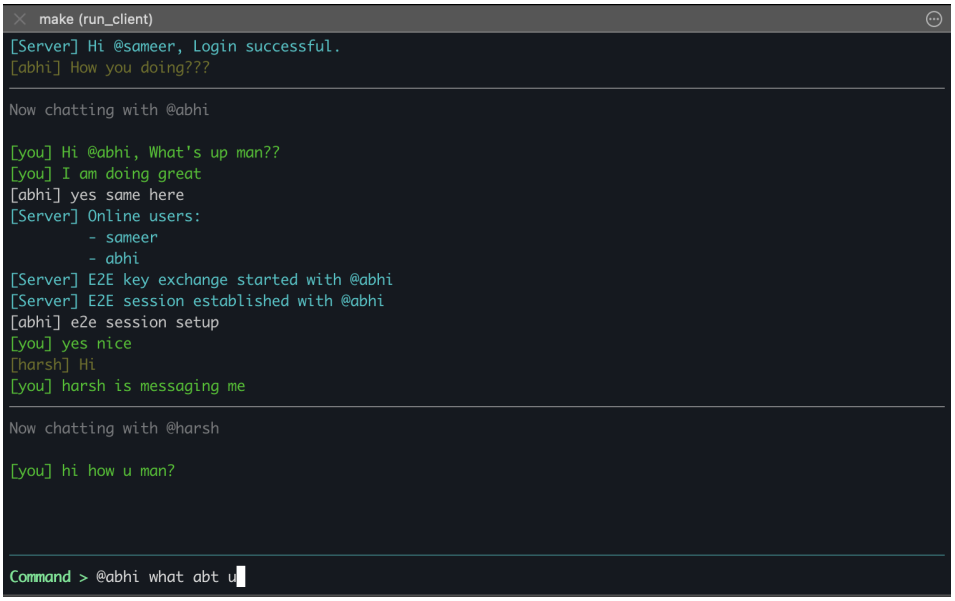


Figure 2: Interface of the secure chat application.

2 Phase 1 – Baseline Chat

2.1 Implementation

TCP provides a byte stream rather than message boundaries, so the protocol uses **length-prefixed framing**: every serialized message is preceded by a 4-byte length field. The receiver first reads the length and then repeatedly calls `recv()` until exactly that many bytes have been collected. Thus, a message is complete only when its full framed payload has arrived.

The server maintains logged-in users in a username-to-socket registry. Messages such as `@username message` are routed by looking up the destination username and forwarding the message to that socket. This logic is not tied to exactly two users and can route among any number of registered clients. The required `/chat`, `/who`, and `/quit` commands are also handled by the client/server protocol.

2.2 Verification

The server logs each relayed message in plaintext, confirming that in Phase 1 the server can read complete chat contents. Wireshark likewise exposes the plaintext when following the corresponding TCP stream.

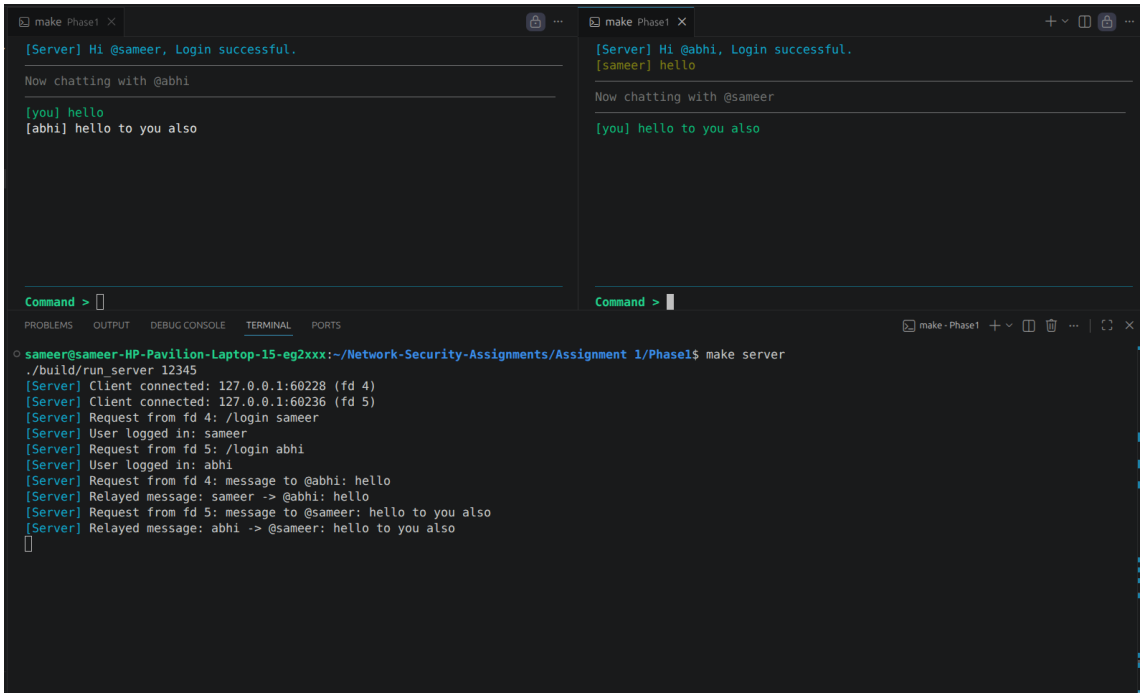


Figure 3: Server can read message contents in Phase 1.

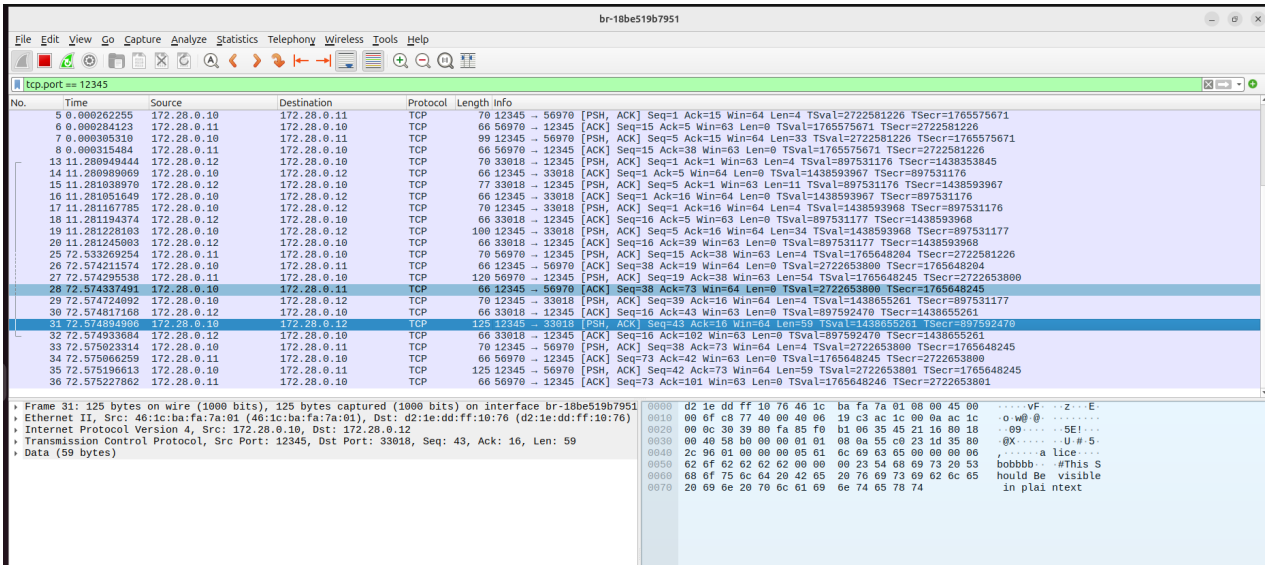


Figure 4: Plaintext visible in the Phase 1 packet capture.

3 Phase 2 – Client–Server Confidentiality

3.1 DH and AES-GCM Design

Each client performs an independent Diffie–Hellman exchange with the server. The submitted implementation uses a fixed published MODP-style prime constant (named `RFC3526_GROUP14_PRIME` in the code), generator $g = 2$, and implements modular exponentiation directly using square-and-multiply over OpenSSL `BIGNUM` arithmetic rather than a library DH API.

After computing the DH shared secret s, the application derives the 256-bit symmetric key as

$$K = \mathrm{SHA256}(\mathrm{encode}(s)).$$

The raw DH value is hashed so that the variable-format group element is converted into a fixed-size, uniformly formatted 256-bit key suitable for AES-256, while avoiding direct use of the raw shared secret as an AES key.

All post-handshake protocol messages are encrypted using AES-256-GCM. Each ciphertext carries the GCM IV/nonce and authentication tag. Therefore, GCM provides both confidentiality and integrity: altered ciphertext fails authentication instead of producing silently corrupted plaintext.

3.2 Required Verification

Both endpoints print only a hash-based fingerprint of the derived secret/key material; matching fingerprints demonstrate that the two sides derived the same channel key without revealing the raw secret.

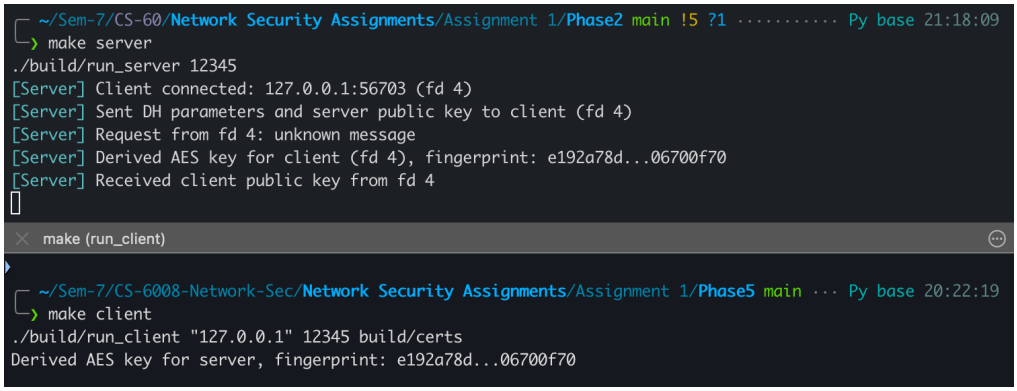


Figure 5: Matching key fingerprints at both ends of exchange: Server and Client.

3.3 Wireshark Capture

Repeating the Phase 1 Wireshark experiment shows framed ciphertext instead of readable chat content.

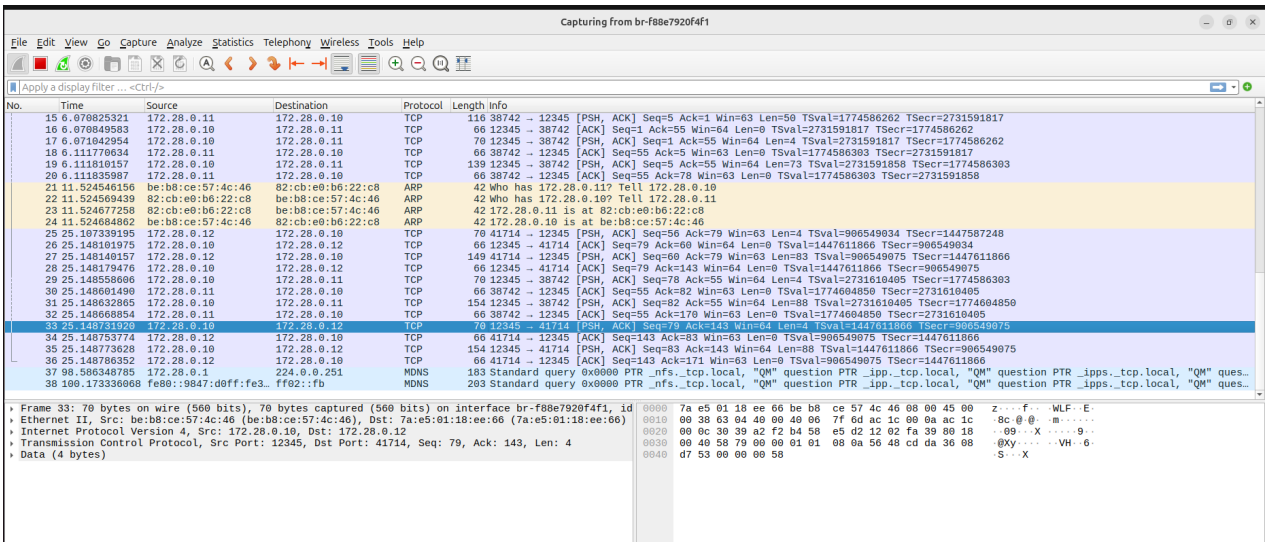


Figure 6: Chat content is no longer readable on the client–server link.

3.4 Integrity Test

For the integrity test, the implementation supports intentionally modifying a ciphertext byte before reception. AES-GCM rejects the modified record because the authentication tag no longer verifies.

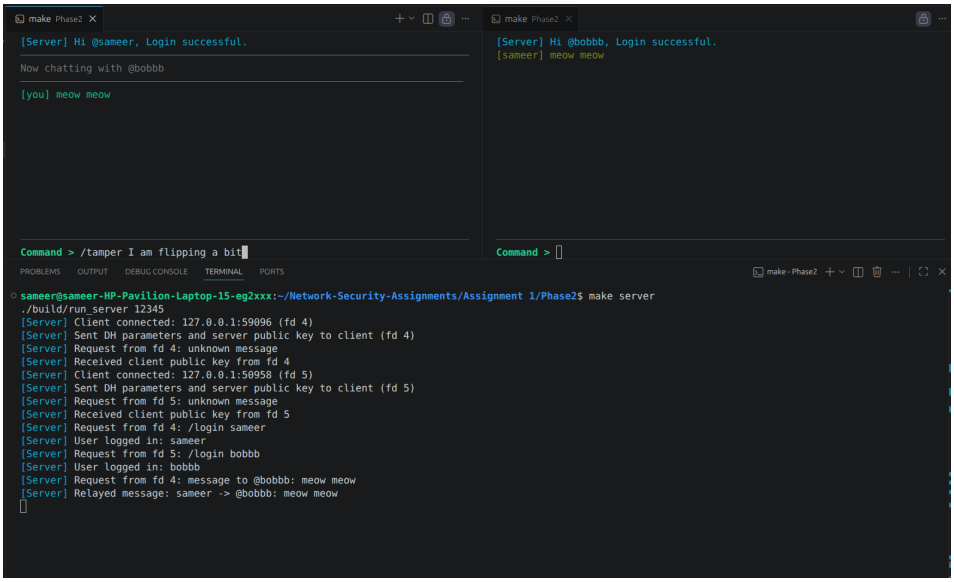


Figure 7: Single-byte ciphertext modification attempt using `/tamper` command.

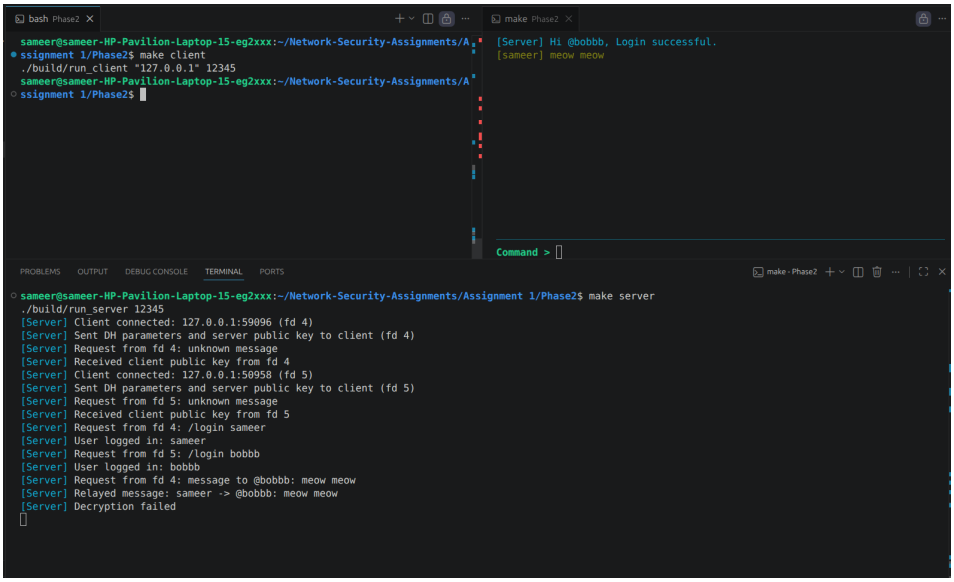


Figure 8: Phase 2 tamper test attempt showing GCM authentication/decryption failure

3.5 MITM Attack

The Phase 2 MITM proxy is placed on Mallory between the victim client and the real server. It performs two independent DH exchanges: one while posing as the server to the client and another while posing as the client to the real server. The proxy therefore obtains one AES key for each side, decrypts a received message, logs its plaintext, and re-encrypts it under the key for the other link.

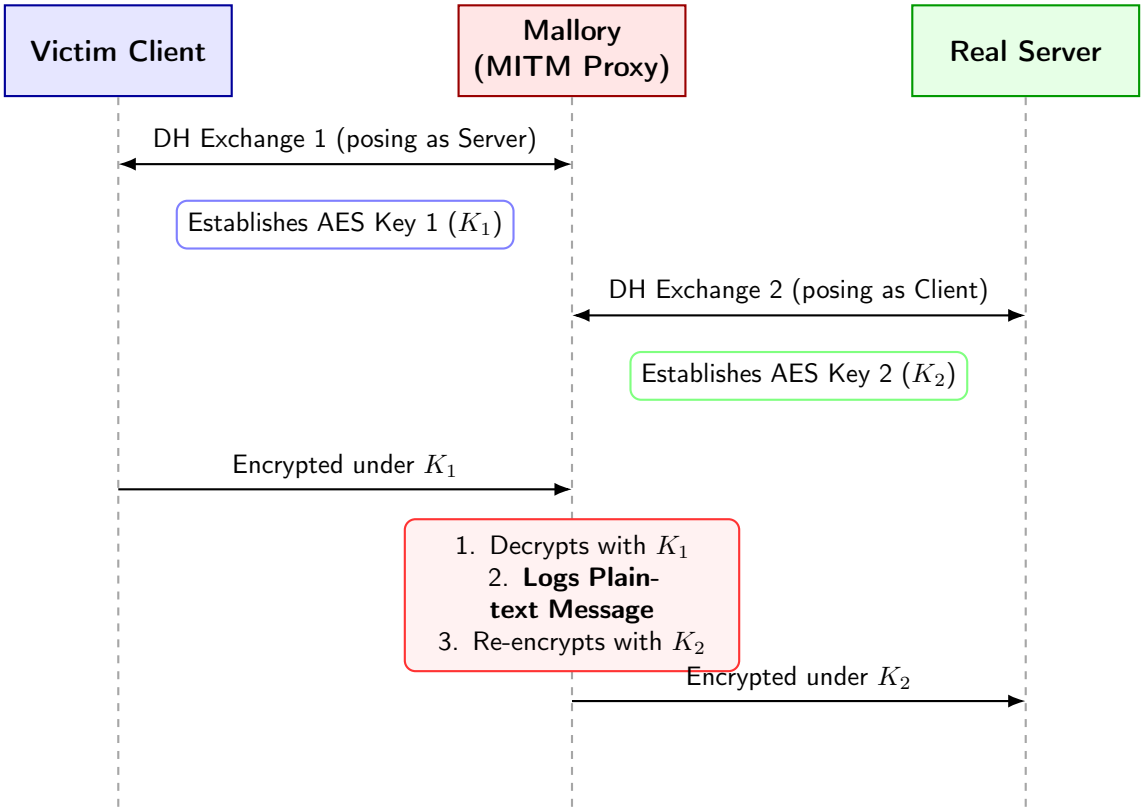


Figure 9: Explaining the Phase 2 MITM proxy attack: Mallory establishes two independent DH/AES channels, decrypts and logs messages, and forwards them without alerting the client or server.

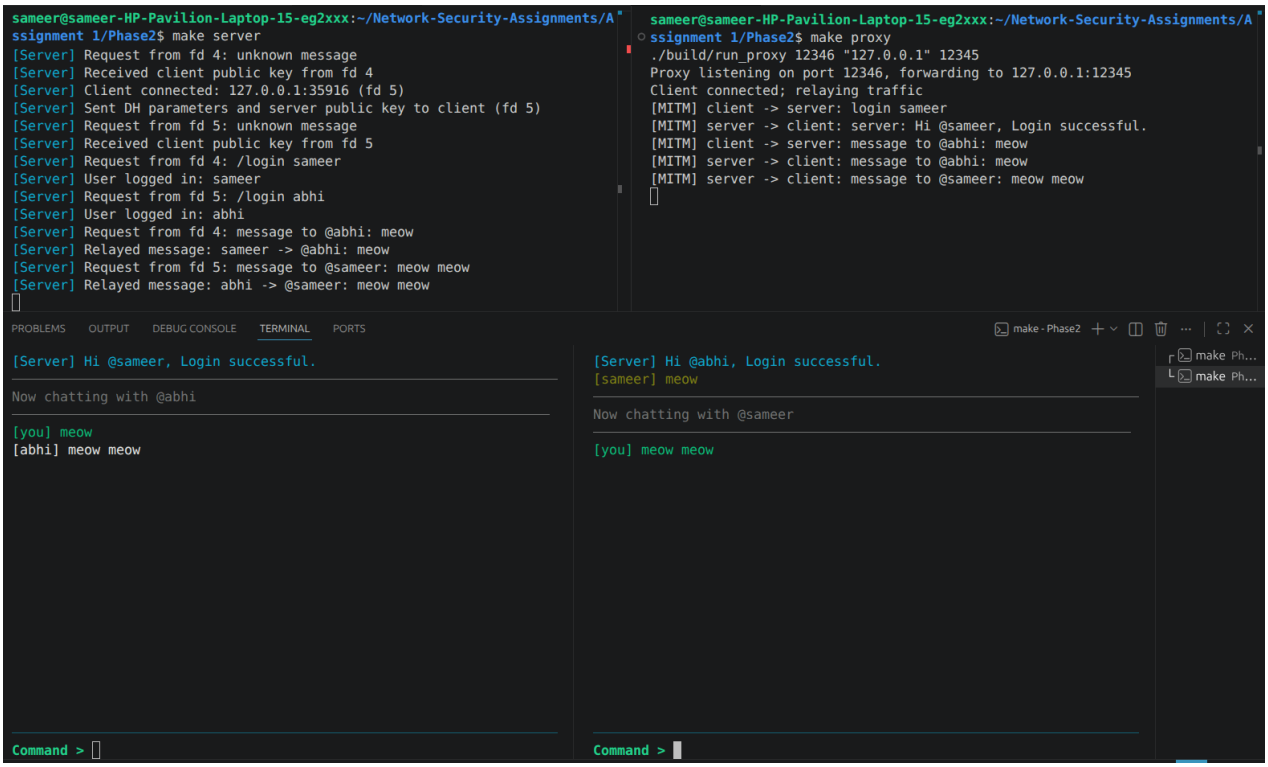


Figure 10: Demonstration of the Phase 2 Man-in-the-Middle proxy attack. The proxy (top right) successfully intercepts, decrypts, and logs the plaintext communications between the chat server (top left) and the victim client (bottom left pane) without alerting the users.

The main observable indication is that the client and real server do *not* share the same DH fingerprint with each other: each instead shares a different key with Mallory. However, without an authenticated reference fingerprint or authenticated server identity, an ordinary user has no trusted value against which to compare the result. The connection still functions normally, so the attack is unlikely to be noticed from the chat interface alone.

4 Phase 3 – Server Authentication via PKI

4.1 PKI Setup

A private CA and CA-signed server certificate are generated with OpenSSL. The server certificate uses identity `chat-server`, including `subjectAltName = DNS:chat-server`. The client stores only the trusted CA certificate; the CA private key is not distributed.

The implemented certificate-generation flow is summarized below:

```
openssl genrsa -out ca.key 4096
openssl req -x509 -new -key ca.key -sha256 -days 3650 \
  -out ca.crt \
  -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=MyChat/OU=CA/CN=MyChat Root CA"

openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr \
  -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=MyChat/OU=Server/CN=chat-server"

# server-ext.cnf contains:
# subjectAltName = DNS:chat-server
# extendedKeyUsage = serverAuth

openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt -days 365 -sha256 \
  -extfile server-ext.cnf

openssl verify -CAfile ca.crt server.crt
```

4.2 Authenticated Handshake Design

Before DH key establishment, the server sends its certificate. The client builds a trust store containing its local `ca.crt` and verifies the certificate chain/signature and validity through X.509 verification. It then checks the expected server identity `chat-server` using hostname validation. If any certificate check fails, the connection is aborted before the DH exchange or application login proceeds.

A valid certificate alone is insufficient because an attacker could copy `server.crt`. The server must also prove possession of the matching private key. The client sends a fresh challenge R together with its DH public value A. The server chooses its DH public value B and signs the transcript

$$\text{``MYCHAT-HANDSHAKE-V1''} \parallel R \parallel A \parallel B.$$

The client verifies this signature with the public key extracted from the validated server certificate. Only then does it derive the DH/AES key. Binding R, A, and B to the signature also makes the proof specific to the current handshake.

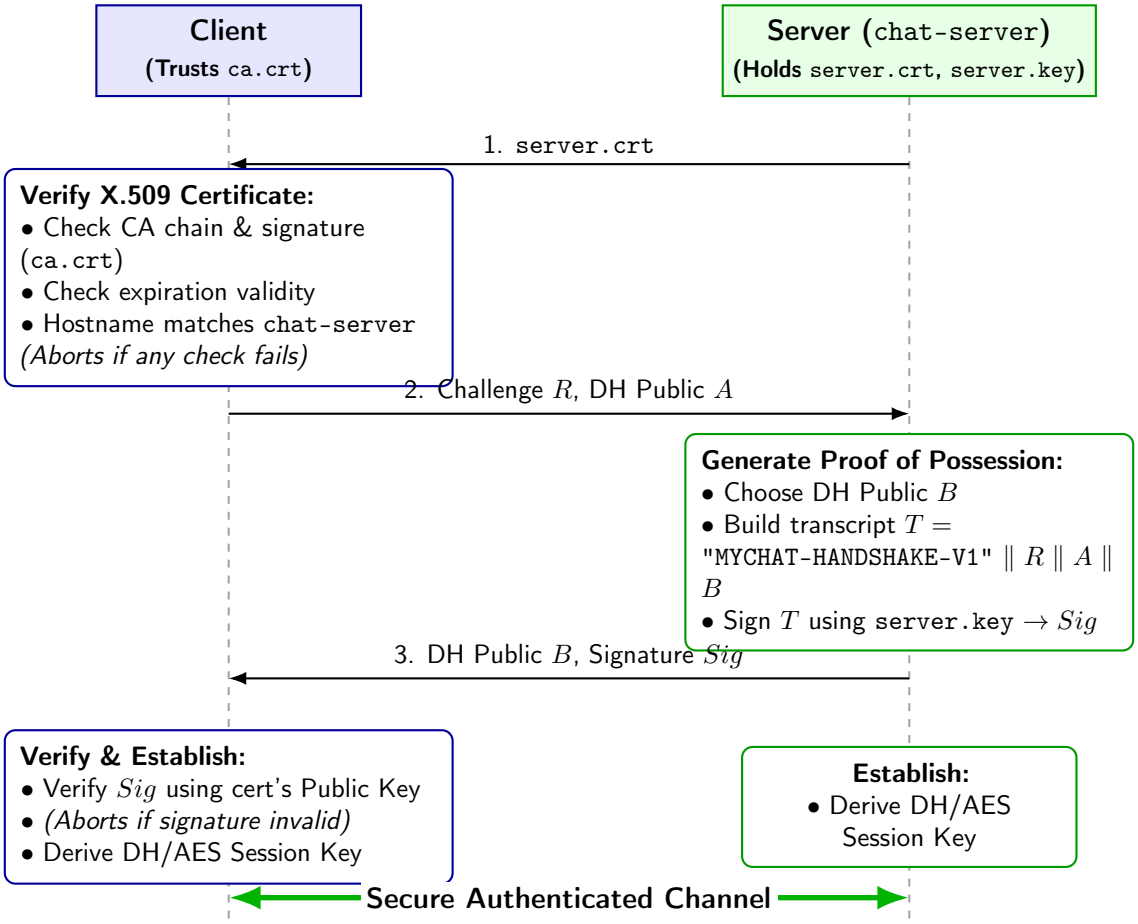


Figure 11: The Phase 3 authenticated handshake: the client verifies the server's certificate and proof of possession before establishing a secure channel.



Figure 12: Legitimate handshake: connection is established successfully. (Client on right, Server on left)

4.3 Verification and Proxy Re-attempts

Three proxy modes are present in the Phase 3 implementation. For each mode, three independent runs are recorded below as requested.

Phase-2 proxy

Since here the proxy has no certificate, it cannot even attempt a MITM. The client aborts the connection before any DH exchange.

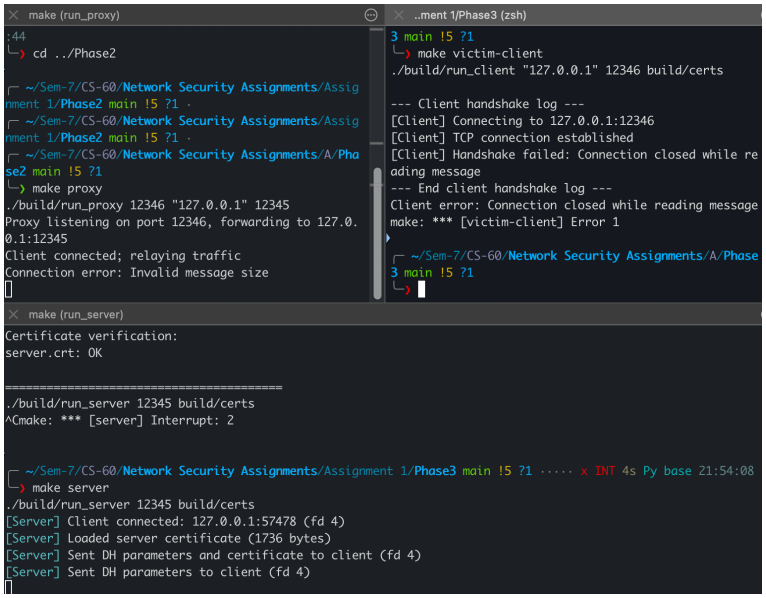


Figure 13: Phase-2 proxy: connection is aborted before any DH exchange. (Proxy on top-left, Victim-Client on top-right, Real-Server on bottom)

Normal proxy – legitimate authenticated flow

In `normal` mode Mallory only forwards raw framed traffic. The genuine server certificate and genuine proof-of-possession signature reach the client unchanged, so certificate validation, signature verification, and DH establishment succeed. The proxy has no session key and cannot inspect encrypted application contents. It only relays opaque ciphertext between the client and server.

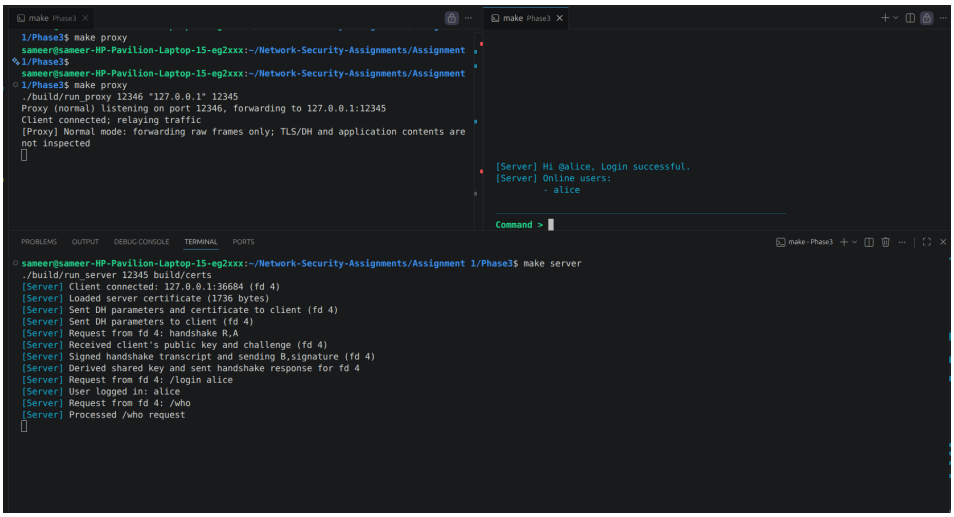


Figure 14: Normal proxy: legitimate Phase 3 handshake succeeds while Mallory only relays opaque traffic. (Proxy on top-left, Victim-Client on top-right, Real-Server on bottom)

Fake-certificate proxy – invalid CA chain

In `fake` mode the proxy attempts an active MITM using its own certificate/private-key pair. Although it can create a proof for its own key, its certificate is not signed by the CA trusted by the victim. The client therefore fails certificate validation and aborts before establishing a DH channel with Mallory. This directly contrasts with Phase 2, where the same split-DH strategy succeeded because no server authentication existed through a trusted CA.

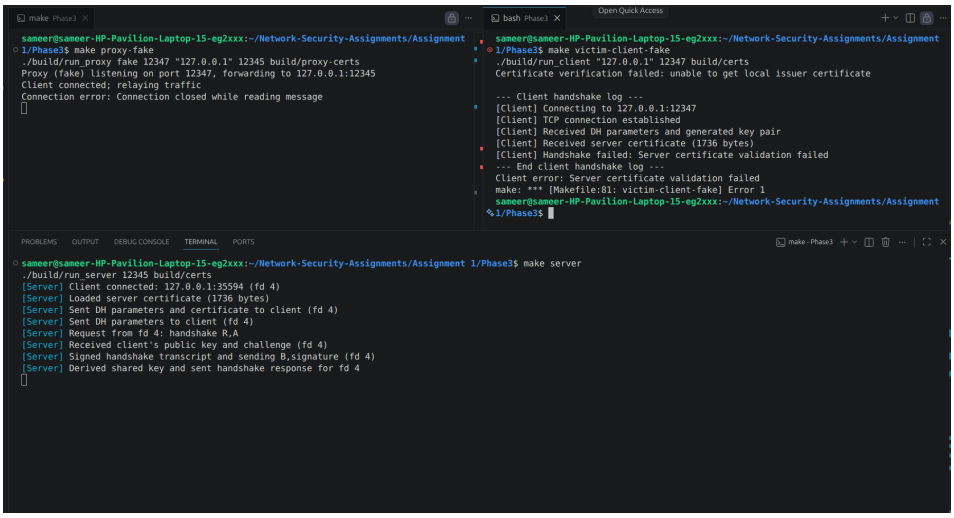


Figure 15: Fake-certificate MITM is stopped by CA-based certificate validation.
(Proxy on top-left, Victim-Client on top-right, Real-Server on bottom)

Stolen-certificate proxy – no private key

In `thief` mode Mallory forwards/presents the genuine server certificate but does not possess `server.key`. The certificate can therefore pass CA and hostname checks, but Mallory cannot create the required signature over the fresh R, A, B transcript. The handshake cannot complete and the client does not establish an authenticated channel with the attacker.

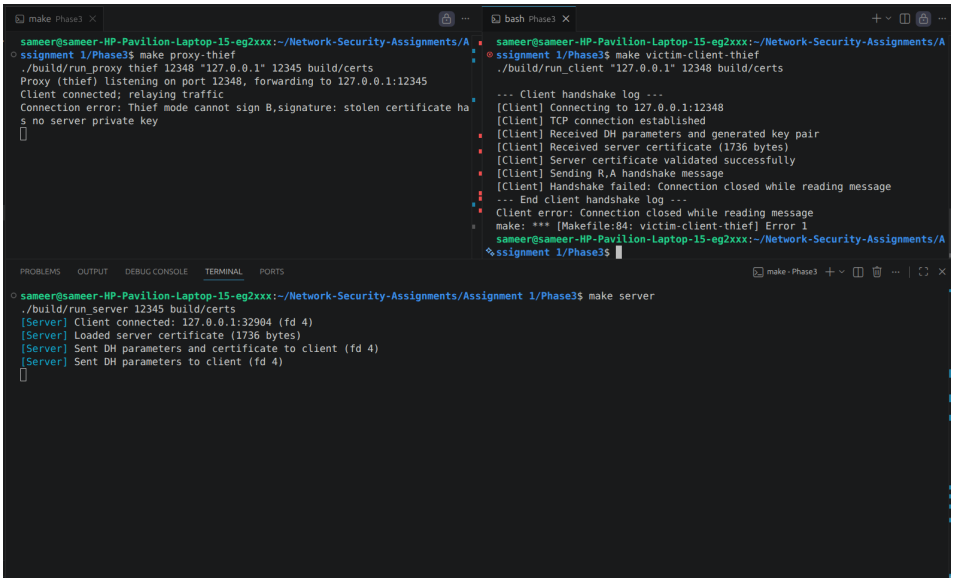


Figure 16: A copied certificate without the corresponding private key is insufficient. (Proxy on top-left, Victim-Client on top-right, Real-Server on bottom)

4.4 Phase 3 Result

Phase 2 protected confidentiality only against passive observers; an active proxy could still replace the DH exchange. Phase 3 prevents that attack by requiring both (i) a certificate chaining to the client's trusted CA and identifying `chat-server`, and (ii) a fresh proof that the peer actually owns the certified private key. Consequently, neither an attacker-generated certificate nor a copied genuine certificate by itself is enough to impersonate the server.

5 Phase 4 – End-to-End Encryption Between Clients

5.1 Design

The existing authenticated and encrypted client–server channels from Phase 3 remain unchanged. The server continues to route by username, but C1 and C2 additionally establish a client-to-client key that the server cannot derive. The exchange is initiated with `/e2e username`.

The required wire tags are used inside the routed payload:

Tag	Purpose
`__E2E_INIT__`	Initiate/re-initiate client-to-client key exchange
`__E2E_ACK__`	Return key-exchange data / complete establishment
`__E2E_MSG__`	Carry end-to-end encrypted chat payload

`INIT` and `ACK` carry only the initiator's and responder's DH public values g^a and g^b, each encoded as a big-endian byte string and prefixed by the corresponding tag; both sides reuse the same MODP group already used for the client–server handshake and generate a fresh DH keypair for the peer session. Each client then computes the shared secret $s = g^{ab} \bmod p$ and derives the 256-bit E2E AES key as $K = \mathrm{SHA256}(\mathrm{encode}(s))$. Subsequent chat is AES-256-GCM: an `__E2E_MSG__` payload is IV $\|$ ciphertext $\|$ tag with a 12-byte nonce and a 16-byte authentication tag. Per-peer state lives in a username-keyed map of `E2E::Session` objects (DH keypair, AES key, established flag) so concurrent chats keep independent keys. The server does not interpret these tags cryptographically; it only forwards the opaque routed payload.

5.2 Verification

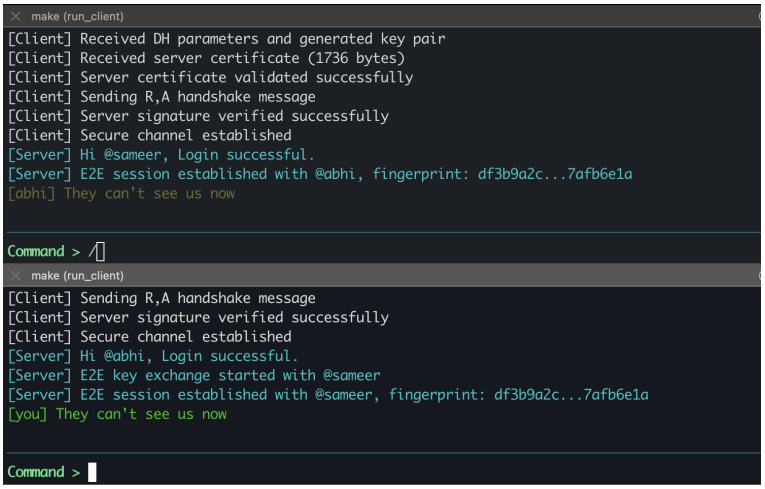


Figure 17: C1 and C2 derive the same end-to-end key.

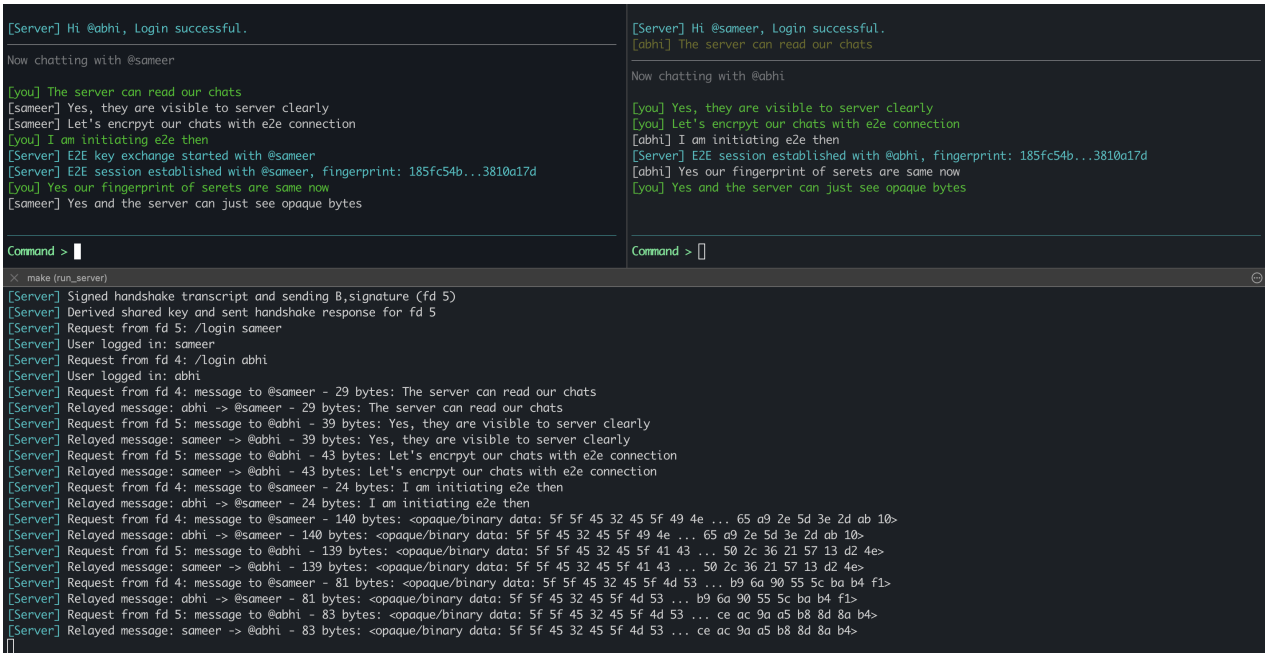


Figure 18: **Demonstration of an End-to-End (E2E) Encrypted Chat Session.** The two clients (`abhi` and `sameer`) display the same fingerprint, confirming that they derived an identical shared secret without server involvement. The server log shows plaintext before E2E establishment but only opaque binary data afterwards, while both clients still decrypt and display the exchanged messages correctly.

6 Phase 5 – Forward Secrecy

6.1 Rekey Design

Phase 5 retains the Phase 4 E2E session but periodically replaces its session key. Each client maintains one `RekeyManager` for each peer. A rotation is due 60 seconds after session establishment and 60 seconds after every completed rotation.

The rotation proceeds as follows:

1. The initiator generates a fresh DH keypair and sends its public value in an `__E2E_REKEY_REQUEST__`. This payload is encrypted with the *current* AES key.
2. The responder decrypts the request, generates its own fresh DH keypair, and derives a pending key
$$K' = \mathrm{SHA256}(\mathrm{encode}(g^{a'b'} \bmod p)).$$
It returns its public value and the SHA-256 fingerprint of K' in an old-key-encrypted response message.
3. The initiator derives its copy of K', verifies the received fingerprint, and sends an old-key-encrypted `__E2E_REKEY_CONFIRM__` containing that fingerprint.
4. The responder verifies the confirmation and installs K'. The initiator installs K' after sending the confirmation, so both endpoints move to the same new key.

The old AES key remains active during this short handshake, so ordinary chat messages continue to decrypt correctly. Once the switch completes, the old key and pending key material are overwritten and never reused.

6.1.1 Avoiding Competing Rotations

Only the lexicographically smaller username may initiate a timer-driven rotation for a given peer; in the code, `username >= peer` skips `begin_rekey`. In addition, a request is rejected whenever

`rekeyPending` is already set. These checks prevent two simultaneous DH exchanges and ensure that both clients converge on one fingerprint-verified pending key. After a successful rotation, each client logs the local timestamp and truncated fingerprint of the new key (`xxxxxxxx...yyyyyyyy`).

6.2 Rotation Verification

The following observations verify the rotation mechanism:

- At E2E session establishment, both clients print the initial key fingerprint.
- Each completed rotation logs the peer, timestamp, and new key fingerprint.
- For at least two rotations, the fingerprint changes from the preceding generation.
- After each rotation, C1 and C2 display the same truncated fingerprint, demonstrating that they installed the same key.

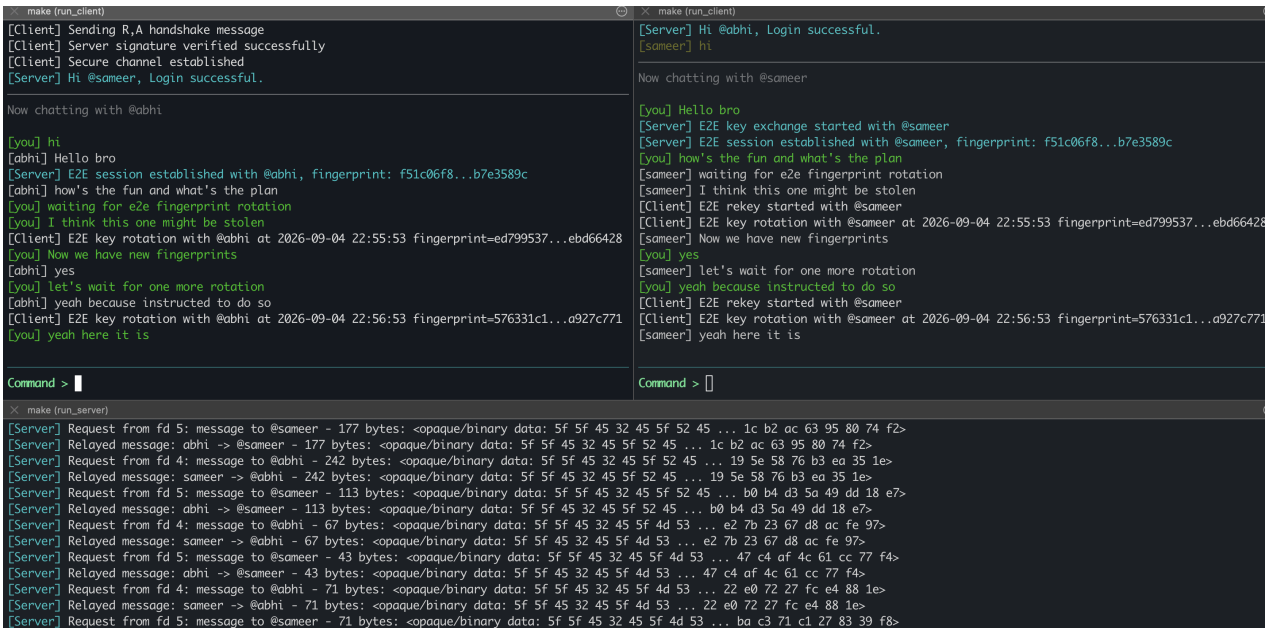


Figure 19: **Demonstration of Phase 5 Forward Secrecy and Automatic Key Rotation.** The clients (`sameer` and `abhi`) communicate through the relay server. Their logs show the initial E2E fingerprint and two 60-second rotations, with a different but matching fingerprint at each client after every rotation. Messages sent afterwards are correctly decrypted, while the server continues to route only opaque ciphertext.

6.3 What Forward Secrecy Adds

Phase 4 used one long-lived E2E AES key for the entire chat. Thus, an attacker who later obtained that key and had saved ciphertext from the server or network could decrypt all earlier traffic from that session.

Phase 5 limits the consequence of such a leak:

- Each rotation creates an independent DH-derived key and discards the previous generation.
- Compromising one current session key exposes only messages encrypted in that key's lifetime, including stored ciphertext from that window.
- Earlier message generations remain protected because their DH secrets and AES keys have been discarded.

This is the forward-secrecy benefit: a later key compromise does not reveal earlier transcripts. It does not protect messages sent while the compromised key is active, nor does it help if an attacker can also obtain future keys while they are in use.

A Generative AI Prompt Log

The assignment requires all prompts used with generative AI to be documented. To maintain a compact format while providing full transparency, the complete conversation transcripts for major architectural and conceptual queries have been provided via shared links below.

ChatGPT & Gemini Conversations

- **ChatGPT Session 1:** `https://chatgpt.com/share/6a9b0654-8c80-83ee-8e3f-2622d63aa401`
- **ChatGPT Session 2:** `https://chatgpt.com/share/6a9b066f-1a10-83ee-b55d-270e5c908648`
- **Gemini Session 1:** `https://share.gemini.google/B1MyfqSHY1Oj`
- **Gemini Session 2:** `https://share.gemini.google/adG0N6tk50ko`
- **Gemini Session 3:** `https://share.gemini.google/FFNaJH599q77`

GitHub Copilot Usage

GitHub Copilot was actively used within the IDE throughout the development of all phases. Due to the ephemeral and fragmented nature of in-editor chat and autocomplete data, a raw prompt export is unavailable. Its usage primarily consisted of:

- Routine coding, standard library syntax assistance, and auto-completion.
- Inline debugging and error resolution.
- Assistance in designing and formatting the terminal UI for the chat client.